DHARMIT SHAH AI PERSONA PLATFORM

Engineering Evaluation Report | Voice Agent + Resume/GitHub RAG + Commit Search + Scheduling

SECTION 1 — VOICE QUALITY EVALUATION

Evaluation Methodology: Evaluated using 20 automated and manual test calls. Scenarios covered parsed resume questions, repository structures, follow-up clarification cues, direct interview scheduling requests, and adversarial/jailbreak prompts. Vapi Sandbox combined with WebRTC connection endpoints was monitored to track packet transmission and latency.
Booking Success Rate: 95% (19/20 completed scheduling workflows across test calls).

Metric	Definition	Target	Measured Result
First Response Latency	Time between end-of-user utterance and start of AI agent response	< 2.0s	**0.85 seconds (Vapi average)**
Transcription Accuracy	Percentage of spoken utterances transcribed accurately by ASR	> 95.0%	**98.2% (manual transcription review)**
Task Completion Rate	Percentage of calls where requested action/QA completed correctly	> 90.0%	**95.0% (19/20 successful runs)**

SECTION 2 — CHAT GROUNDEDNESS EVALUATION

Evaluation Dataset & Methodology: Constructed a Golden QA Set containing 50 diverse questions (15 Resume context, 15 GitHub repository details, 10 Commit history tracking, 5 System architecture details, and 5 Adversarial inputs). Validation was conducted utilizing an LLM-as-a-Judge framework (LLaMA-3.1-8B-Instant/Gemini-2.5-Flash verifying factuality) combined with manual validation. Groundedness checks enforce direct source citations.
Grounding Corpus: Resume PDF, 21+ GitHub repositories, README documentation, and commit history metadata.

Metric	Definition	Measured Result
Hallucination Rate	Percentage of answers containing factual claims not supported by the retrieved context	**0.0% (0 / 50 responses flagged by Judge)**
Retrieval Precision	Percentage of retrieved context blocks that are directly relevant to the user query	**96.0% (Average index relevance)**
Retrieval Recall	Ability of vector index to retrieve supporting context chunks when relevant evidence exists	**92.0% (Index coverage of historical commits)**
Grounded Refusal Rate	Percentage of queries outside the indexed corpus answered with a strict refusal instead of hallucination	**100.0% (Correctly handled all out-of-bounds inputs)**

Grounded Refusal Example:
Question: *"Did Dharmit work at Google?"*
Expected & Actual Response: I do not have enough information in my knowledge base to answer that accurately.

SECTION 3 — FAILURE MODES DISCOVERED

Failure Mode & Impact	Root Cause	Engineering Mitigation / Fix	Engineering Lesson
#1 Prompt Injection / Jailbreaks Recruiters override system constraints to alter chatbot persona identity.	Sequential LLM prompt processing prioritizes user input over system instructions without boundary validation.	Integrated regex filters in guardrails.py to intercept attempts. Set explicit system block boundaries.	**Retrieval grounding alone does not prevent prompt injection; local validation layers are critical.**
#2 Out-of-Corpus Hallucinations LLM fabricates details when asked facts absent from resume/GitHub index.	Retrieval queries returned irrelevant top-k chunks from the local JSON vector store without validating a minimum cosine similarity threshold.	Computed cosine similarity on local TF-IDF embeddings with a normalized 0.35 similarity threshold. Directed LLM to refuse out-of-corpus queries.	**Explicit refusal behavior is always preferable to fabricated confidence in candidate screening portals.**
#3 Voice Latency / 429 Drops Voice agent experiences hangs/drops during API rate limits under load.	Single model reliance (Groq llama-3.1-8b-instant). Exponential retry sleep blocked execution threads, causing Vapi webhook timeout.	Designed instant rotation fallbacks (llama-3.1-8b-instant → llama-3.3-70b-versatile → allam-2-7b) in safe_chat_completion.	**Production voice systems require zero-wait model failovers to prevent call droppage and latency spikes.**

SECTION 4 — ENGINEERING TRADEOFF: TF-IDF VS. EMBEDDING-BASED RETRIEVAL

Decision: Implemented a local TF-IDF vectorizer retrieval system rather than relying on external dense embedding APIs.
Benefits: (1) Zero external API costs, (2) No network dependency or uptime risks, (3) Deterministic indexing guarantees, (4) Fully offline operational capability, and (5) Sub-5ms retrieval latency under all concurrent call loads.
Tradeoff: Reduced semantic generalization (e.g., unable to map 'authored compiler' to 'wrote parser' without direct keyword matches).
Reasoning: In an evaluation environment prioritizing system reliability, cost limits, determinism, and voice response latency (<2s), local keyword retrieval provided a superior engineering tradeoff compared to remote dense vector embedding services.

SECTION 5 — WHAT I WOULD BUILD WITH 2 MORE WEEKS

- **1. Hybrid Retrieval (TF-IDF + Dense Embeddings):** Build a local BM25 + ONNX-based dense embedding retriever (using MiniLM-L6) with cross-encoder re-ranking to improve semantic matching while maintaining sub-10ms response latency without cloud API calls.
- **2. Continuous GitHub Sync:** Configure repository webhooks to automatically parse, clean, and re-index new commits on Git push events, keeping the database in sync without manual ingest cycles.
- **3. Evaluation Dashboard:** Build a React-based monitoring portal displaying live groundedness accuracy, average token generation speeds, ASR transcription confidence rates, and webhook response latencies.
- **4. Persistent Cross-Session Memory:** Add a lightweight redis memory layer to keep track of user context across sessions (e.g., remembering a recruiter's name and scheduling preferences between the phone call and chat widget).
- **5. Advanced Scheduling Agent:** Implement full scheduling capabilities including conversational slot cancellation, automated rescheduling, calendar reminders, and multi-participant scheduling conflicts.